

28.09.2026

Nome: Júlia Barreira de Carvalho

Disciplina: Estrutura de Dados II;

Professora: Profa. Kadidja Valéria

Modalidade: individual, remota e assíncrona

Tempo estimado: 1 hora e 45 minutos

Etapa 1 — Revisão bibliográfica

Consulte as referências bibliográficas indicadas no plano de ensino e os materiais disponibilizados na disciplina. Inicie pelos conceitos básicos de árvore; depois estude árvore geral, árvore binária, árvore binária de busca (ABB), AVL, rubro-negra, B e B+.

Para cada estrutura, procure identificar: como os dados são organizados; qual propriedade deve ser mantida; como funcionam busca e inserção; que ajustes podem ocorrer após alterações; e em que contexto ela é útil. Redija com suas próprias palavras.

01. Conceitos gerais

Nó: Unidade fundamental da árvore que armazena um dado (chave ou valor) e referências para seus nós adjacentes (filhos ou pai).

Raiz: O único nó da árvore que não possui pai; é o ponto de entrada superior da estrutura.

Pai e Filho: Relação hierárquica direta entre um nó e seus sucessores imediatos (filhos) ou antecessor (pai). Nós que compartilham o mesmo pai são chamados de irmãos.

Folha (ou terminal): Nó que não possui nenhum filho (grau zero).

Altura: O comprimento do caminho mais longo da raiz até uma folha, ou o número de arestas nesse caminho (alguns autores contam o número de nós).

Percurso: Forma de visitar todos os nós da árvore de maneira sistemática. Os principais percursos em árvores binárias são:

- **Pré-ordem:** Visita a raiz, depois a subárvore esquerda e a subárvore direita.
- **Em ordem (In-ordem):** Visita a subárvore esquerda, depois a raiz e por fim a subárvore direita (essencial em ABBs para retornar dados ordenados).
- **Pós-ordem:** Visita a subárvore esquerda, depois a subárvore direita e por último a raiz.
- **Em largura (por níveis):** Visita os nós nível por nível, da esquerda para a direita.

02. Árvore Geral

Como os dados são organizados: Os nós são organizados em uma hierarquia de pais e filhos, onde cada nó pode possuir um número arbitrário (ilimitado) de filhos, sem restrição estrita sobre o grau máximo de ramificação.

Propriedade mantida: Relação hierárquica estrita (ancestral/descendente) onde cada nó (exceto a raiz) possui exatamente um pai.

Busca e Inserção: A busca pode ser feita por percursos genéricos (como profundidade ou largura) percorrendo os ramos até encontrar a chave. A inserção requer a especificação do nó pai ao qual o novo nó será vinculado como filho. Como não há balanceamento automático por padrão, a eficiência depende da forma como a árvore cresce.

Ajustes: Geralmente não há ajustes automáticos de reestruturação por rotação; o gerenciamento estrutural depende da lógica da aplicação.

Contexto de utilidade: Sistemas de arquivos (diretórios e arquivos), representação de documentos estruturados (como DOM em HTML/XML) e organogramas empresariais.

03. Árvore Binária

Como os dados são organizados: Semelhante à árvore geral, mas com uma restrição rígida: cada nó possui no máximo dois filhos, convencionalmente denominados *filho esquerdo* e *filho direito*.

Propriedade mantida: O grau de saída de qualquer nó é ≤ 2 . Pode ser classificada como cheia, estritamente binária ou completa.

Busca e Inserção: A busca no caso geral exige a varredura exaustiva de ambos os ramos (complexidade $O(N)$), a menos que siga regras adicionais de ordenação. A inserção preenche os espaços vazios conforme a lógica estrutural adotada (por exemplo, preenchendo o primeiro nível disponível da esquerda para a direita).

Ajustes: Reestruturações pontuais durante inserções dependem do tipo específico de árvore binária (por exemplo, manutenção de completude em heaps).

Contexto de utilidade: Expressões aritméticas (árvores de expressão onde operadores são nós internos e operandos são folhas) e algoritmos de divisão e conquista.

04. Árvore Binária de Busca (ABB)

Como os dados são organizados: Organização baseada em comparação de chaves. Para qualquer nó X , todos os valores na subárvore esquerda são estritamente menores que a chave de X , e todos os valores na subárvore direita são maiores ou iguais.

Propriedade mantida: Invariante da ABB:

$chave(sub.esquerda) < chave(X) < chave(sub.direita)$.

Busca e Inserção: A busca compara a chave procurada com o nó atual, descendo para a esquerda ou para a direita recursivamente (complexidade média $O(\log N)$). A inserção localiza a posição correta seguindo a mesma lógica de comparação até encontrar uma folha vazia para anexar o novo nó.

Ajustes: Nenhuma rotação é aplicada por padrão. Se inserções ocorrerem de forma ordenada, a ABB pode degenerar em uma estrutura linear (tipo lista encadeada), degradando a complexidade para $O(N)$.

Contexto de utilidade: Dicionários em memória, tabelas de símbolos básicas e conjuntos ordenados onde operações dinâmicas frequentes são necessárias, assumindo entrada razoavelmente balanceada.

05. Árvore AVL

Como os dados são organizados: Organização baseada em comparação de chaves. Para qualquer nó X , todos os valores na subárvore esquerda são estritamente menores que a chave de X , e todos os valores na subárvore direita são maiores ou iguais.

Propriedade mantida: Invariante da ABB:

$chave(sub.esquerda) < chave(X) < chave(sub.direita)$.

Busca e Inserção: A busca compara a chave procurada com o nó atual, descendo para a esquerda ou para a direita recursivamente (complexidade média $O(\log N)$). A inserção localiza a posição correta seguindo a mesma lógica de comparação até encontrar uma folha vazia para anexar o novo nó.

Ajustes: Nenhuma rotação é aplicada por padrão. Se inserções ocorrerem de forma ordenada, a ABB pode degenerar em uma estrutura linear (tipo lista encadeada), degradando a complexidade para $O(N)$.

Contexto de utilidade: Dicionários em memória, tabelas de símbolos básicas e conjuntos ordenados onde operações dinâmicas frequentes são necessárias, assumindo entrada razoavelmente balanceada.

06. Árvore Rubro-Negro (Red-Black Tree)

Como os dados são organizados: Uma árvore binária de busca auto-balanceada onde cada nó possui um atributo extra de cor (vermelho ou preto).

Propriedade mantida:

1. Todo nó é vermelho ou preto.
2. A raiz é preta.
3. Todas as folhas (nós nulos/NIL) são pretas.
4. Se um nó for vermelho, seus dois filhos devem ser pretos (sem dois vermelhos consecutivos).
5. Todo caminho de um nó dado até suas folhas descendentes contém o mesmo número de nós pretos (altura preta constante).

Busca e Inserção: A busca segue a regra padrão de ABB ($O(\log N)$). A inserção adiciona um nó como vermelho por padrão e pode violar as propriedades de cores ou vizinhança.

Ajustes: Utiliza **recolorações** de nós e **rotações** estruturais para restaurar as propriedades da árvore de forma eficiente (o que exige menos rotações estritas que a AVL no pior caso).

Contexto de utilidade: Bibliotecas padrão de linguagens para implementação de estruturas associativas (como `TreeMap` e `TreeSet` em Java, ou `std::map` em C++).

07. Árvore B

Como os dados são organizados: Uma árvore de busca balanceada *multicaminho* (projetada para sistemas que leem blocos de dados em memória secundária/disco), onde cada nó pode conter múltiplas chaves e múltiplos ponteiros para filhos.

Propriedade mantida: Um nó de ordem M pode ter no máximo M filhos e conter até $M - 1$ chaves, ordenadas de forma crescente. Exceto a raiz e as folhas, todos os nós possuem pelo menos $\lceil M/2 \rceil$ filhos, garantindo que a árvore permaneça ampla e de altura extremamente baixa.

Busca e Inserção: A busca faz uma busca binária ou sequencial dentro das chaves do nó atual para decidir em qual subfilho descer. A inserção é feita sempre nas folhas; se um nó folha ultrapassa o limite de chaves, ele sofre **split (divisão)**, promovendo a chave mediana para o nó pai.

Ajustes: Divisões (*splits*) propagando-se para cima quando nós cheios estouram, e fusões (*merges*) ou redistribuições no caso de remoções.

Contexto de utilidade: Sistemas de gerência de banco de dados (SGBDs) e sistemas de arquivos para indexação em discos magnéticos ou SSDs (minimizando acessos I/O).

08. Árvore B+

Como os dados são organizados: Uma evolução da Árvore B onde **todos os dados/registros ou ponteiros para dados residem exclusivamente nas folhas**, enquanto os nós internos armazenam apenas chaves duplicadas para servir como guia de roteamento.

Propriedade mantida: As folhas são interligadas sequencialmente por meio de ponteiros bidirecionais (formando uma lista encadeada nas folhas), mantendo a estrutura balanceada de árvore B nas camadas superiores.

Busca e Inserção: A busca desce obrigatoriamente até as folhas, independentemente de a chave buscada ser intermediária. A inserção ocorre nas folhas com divisão de blocos quando necessário, atualizando os índices separadamente.

Ajustes: Divisão de nós folha e nós internos, mantendo a consistência dos ponteiros sequenciais da camada inferior.

Contexto de utilidade: Índices de bancos de dados relacionais e sistemas de arquivos modernos, sendo ideal tanto para buscas pontuais quanto para **consultas por intervalo** eficientes graças à listagem encadeada das folhas.

09. Heap (Árvore Binária de Mini ou Maximização)

Como os dados são organizados: Uma árvore binária quase completa armazenada de forma compacta (geralmente mapeada em um vetor/array linear), onde os dados são organizados hierarquicamente por prioridade.

Propriedade mantida: Propriedade de heap: em um Max-Heap, qualquer pai é maior ou igual aos seus filhos; em um Min-Heap, qualquer pai é menor ou igual aos seus filhos. A raiz sempre contém o elemento extremo (máximo ou mínimo).

Busca e Inserção: A busca por um elemento arbitrário é custosa ($O(N)$), pois o heap não ordena elementos da esquerda para a direita como a ABB. A inserção adiciona o elemento na primeira posição livre disponível no final da árvore (fim do vetor) e realiza um ajuste para cima (*bubble-up* / *percolate-up*).

Ajustes: Operações de **subida** (*heapify-up*) e **descida** (*heapify-down*) de elementos após inserção ou extração da raiz ($O(\log N)$).

Contexto de utilidade: Implementação de filas de prioridade (*Priority Queues*) e o algoritmo de ordenação *Heapsort*.

10. Trie (Árvore de Prefixos / Árvore Digital)

Como os dados são organizados: Uma árvore de busca indexada onde as chaves são tipicamente strings. Os nós representam caracteres individuais, e os caminhos da raiz até as folhas formam as palavras completas. Prefixo compartilhados utilizam os mesmos nós iniciais.

Propriedade mantida: Os nós filhos de um pai compartilham o mesmo prefixo comum correspondente ao caminho percorrido desde a raiz. Cada nó possui um vetor ou mapa de ponteiros do tamanho do alfabeto.

Busca e Inserção: A busca consome tempo proporcional ao tamanho da chave ($O(M)$ onde M é o comprimento da string), independentemente do número de palavras armazenadas. A inserção percorre os caracteres da string, criando novos nós apenas para os caracteres que ainda não existem na árvore.

Ajustes: Remoção de nós órfãos caso uma palavra seja deletada e seus caracteres não pertençam a nenhuma outra palavra.

Contexto de utilidade: Mecanismos de auto-completar (*autocomplete*), corretores ortográficos, dicionários preditivos e roteamento IP (prefix matching).

Etapa 2 — Quadro comparativo

Preencha todas as linhas pertinentes ao conteúdo da disciplina. Acrescente árvore geral, árvore binária, heap e trie quando forem abordadas nas referências ou aulas. A coluna de referência deve apontar a fonte específica usada para cada linha.

Estrutura	Org. dos dados	Regra ou propriedade principal	Operação ou ajuste importante	Ex. de aplicação	Ref. consultada
Árvore geral	Hierarquia de nós sem limite de filhos por pai.	Cada nó possui um único pai (exceto a raiz).	Inserção por vínculo direto de ponteiro de pai para filho.	Sistemas de arquivos de diretórios.	Notas de aula / Material base da disciplina
Árvore binária	Hierarquia restrita a no máximo dois filhos por nó.	Grau de saída por nó ≤ 2 .	Percursos estruturados (pré, em e pós-ordem).	Árvores de expressão matemática.	Notas de aula / Material base da disciplina
Árvore binária de busca (ABB)	Organizada por comparações de chave.	Sub. esquerda $<$ Nó $<$ Sub. direita.	Busca binária recursiva; risco de degradação linear.	Tabelas de símbolos simples em memória.	Material de apoio de Estrutura de Dados II
AVL	Árvore binária de busca com controle estrito de altura.	Diferença de altura entre subárvores ≤ 1 .	Rotações simples e duplas para rebalanceamento.	Mapeamentos dinâmicos de alta exigência de busca.	Referência bibliográfica oficial da disciplina
Rubro-negra	Árvore binária de busca com atributos de cor (V/P).	Cores alternadas e contagem igual de nós pretos por caminho.	Recolorações e rotações estruturais controladas.	TreeMap / TreeSet na API Java.	Referência bibliográfica oficial da disciplina
B	Multicaminho balanceada para armazenamento em bloco.	Nós contêm múltiplas chaves e M ponteiros; nós internos balanceados.	Operação de <i>split</i> (divisão) de nós cheios.	Bancos de dados relacionais (SGBDs - blocos em disco).	Referência bibliográfica oficial da disciplina

B+	Multicaminho com dados restritos às folhas e encadeamento.	Chaves duplicadas nos índices; dados completos e lista encadeada apenas nas folhas.	Divisão de folhas e manutenção de ponteiros sequenciais.	Índices avançados em bancos de dados (consultas por intervalo).	Referência bibliográfica oficial da disciplina
Heap	Árvore binária quase completa mapeada em vetor.	Pai maior (Max-Heap) ou menor (Min-Heap) que seus filhos.	Operações de <i>heapify</i> (subida e descida).	Filas de prioridade e algoritmo Heapsort.	Material de apoio de Estrutura de Dados II
Trie	Árvore de caracteres organizada por prefixos.	Caminhos corresponde m a prefixos compartilha dos de strings.	Inserção e busca indexada caractere a caractere ($O(M)$).	Sistemas de autocompletar e dicionários preditivos.	Material de apoio de Estrutura de Dados II

Etapa 3 — Identificação por analogias

Para cada situação, identifique a estrutura, justifique sua resposta com uma propriedade técnica e explique um limite da analogia (algo que a comparação não representa fielmente). Evite responder apenas com o nome da árvore.

1. Uma estante de números é reorganizada por rotações quando um lado fica alto demais em relação ao outro.
2. Um catálogo guarda várias chaves por página; quando uma página fica cheia, ela é dividida.
3. Uma fila mantém a tarefa de maior prioridade no topo para retirá-la primeiro.
4. Um índice percorre letras sucessivas e compartilha o início das palavras de mesmo prefixo.
5. Uma estrutura usa cores, recolorações e rotações para manter controlada a altura dos caminhos de busca.
6. Um índice conduz às folhas que contêm os registros, ligadas entre si para facilitar consultas por intervalo.
7. Numa coleção de números, cada nó direciona valores menores para a esquerda e maiores para a direita.

Situação: Uma estante de números é reorganizada por rotações quando um lado fica alto demais em relação ao outro.

- **Estrutura:** Árvore AVL.
- **Justificativa técnica:** A propriedade de balanceamento rígido da AVL exige que a diferença de altura entre as subárvores esquerda e direita de qualquer nó seja no máximo 1. Quando essa regra é quebrada, a estrutura executa rotações locais (simples ou duplas) para restabelecer a estabilidade e garantir busca eficiente em $O(\log N)$.
- **Limite da analogia:** Na realidade computacional, as "rotações" não envolvem mover fisicamente livros/dados na memória, mas sim a reatribuição lógica de referências (ponteiros de nós pais e filhos).

Situação: Um catálogo guarda várias chaves por página; quando uma página fica cheia, ela é dividida.

- **Estrutura:** Árvore B.
- **Justificativa técnica:** Na Árvore B, cada nó comporta múltiplos valores (chaves) e vários ponteiros para filhos. Quando um nó atinge sua capacidade máxima de chaves, ocorre uma operação de divisão (*split*), fragmentando a página e promovendo a chave mediana para o nível superior.
- **Limite da analogia:** As páginas de um livro físico são sequenciais e estáticas em tamanho, enquanto os nós de uma Árvore B ajustam-se dinamicamente e os nós intermediários servem estritamente como guias de roteamento para blocos de dados.

Situação: Uma fila mantém a tarefa de maior prioridade no topo para retirá-la primeiro.

- **Estrutura:** Heap (Max-Heap ou Min-Heap).
- **Justificativa técnica:** O Heap garante que o elemento de maior (ou menor) prioridade ocupe sempre a raiz da árvore binária quase completa. Isso permite a extração imediata do elemento prioritário em tempo $O(1)$, com reorganização rápida ($O(\log N)$) do restante da estrutura.
- **Limite da analogia:** Uma fila cotidiana comum opera estritamente no modelo FIFO (*First In, First Out*), enquanto o Heap prioriza os elementos com base em seu valor numérico/chave, e não estritamente pela ordem de chegada.

Situação: Um índice percorre letras sucessivas e compartilha o início das palavras de mesmo prefixo.

- **Estrutura:** Trie (Árvore de Prefixos).
- **Justificativa técnica:** A Trie organiza as palavras caractere por caractere a partir de nós ramificados na raiz. Palavras que iniciam com a mesma sequência de letras compartilham exatamente o mesmo caminho inicial, economizando redundâncias estruturais e agilizando buscas por prefixo.
- **Limite da analogia:** Em um índice alfabético de livro tradicional, as palavras são listadas em listas sequenciais ou blocos por letra, e não ramificadas nó a nó através de grafos de caracteres individuais.

Situação: Uma estrutura usa cores, recolorações e rotações para manter controlada a altura dos caminhos de busca.

- **Estrutura:** Árvore Rubro-Negra (*Red-Black Tree*).
- **Justificativa técnica:** A Árvore Rubro-Negra utiliza um bit extra de controle de cor associado a cada nó (vermelho ou preto) combinado a regras rígidas de alternância e

balanceamento de caminhos pretos. Isso assegura que nenhum caminho seja mais que o duplo de outro, mantendo a altura controlada de forma menos restrita (e com menos rotações) que a AVL.

- **Limite da analogia:** Cores no mundo real carregam significados semânticos ou visuais (como status ou categorias), enquanto na árvore funcionam puramente como uma regra lógica binária para restabelecer balanceamentos estruturais após inserções e remoções.

Situação: Um índice conduz às folhas que contêm os registros, ligadas entre si para facilitar consultas por intervalo.

- **Estrutura:** Árvore B+.
- **Justificativa técnica:** Na Árvore B+, os nós internos funcionam unicamente como índices de roteamento, enquanto todas as chaves de dados e registros reais residem nas folhas. Além disso, as folhas possuem ponteiros bidirecionais que as interligam sequencialmente, permitindo percorrer intervalos inteiros de forma linear e extremamente rápida.
- **Limite da analogia:** Um índice remissivo de final de livro aponta diretamente para páginas estáticas, mas não possui uma lista encadeada dinâmica automatizada que conecte fisicamente os conceitos contínuos em uma sequência iterativa ininterrupta.

Situação: Numa coleção de números, cada nó direciona valores menores para a esquerda e maiores para a direita.

- **Estrutura:** Árvore Binária de Busca (ABB).
- **Justificativa técnica:** A propriedade fundamental da ABB estabelece que para qualquer nó, o subgalho esquerdo armazena chaves menores e o subgalho direito armazena chaves maiores. Isso define o comportamento de busca por divisão e conquista.
- **Limite da analogia:** Uma coleção física de objetos divididos em caixas esquerda/direita não se auto-reorganiza ou ajusta automaticamente em formato hierárquico quando novos itens são inseridos fora de ordem, correndo o risco de virar uma estrutura totalmente desbalanceada.

Referências Consultadas

- Material didático e notas de aula da disciplina de **Estrutura de Dados II** — Profa. Kadidja Valéria.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. *Algoritmos: Teoria e Prática*. Editora Campus/Elsevier.
- Ziviani, Nivio. *Projeto de Algoritmos com Implementações em Pascal e C*. Cengage Learning.